



Concurrency control (1) - Lock based protocol

✓ Concurrency control

(technique used to implement isolation level)



Outline

- Lock-Based Protocols
- Timestamp-Based Protocols
- Validation-Based Protocols
- Multiple Granularity
- Multiversion Schemes
- Insert and Delete Operations
- Concurrency in Index Structures

- DBMS employ this technique

Lock-Based Protocols (lock-based technique)



Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
 1. **exclusive (X) mode**. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
 2. **shared (S) mode**. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

- there are many mode in locking, but this the most primitive mode (Lock primitive - instruction used to lock data item)

Lock based technique comprised of 3 parts

1. 🌱 **Lock primitives:** instruction used to lock(protect) resources
 - X Lock → data item can be both read as well as written (if you want to update too)
 - S Lock → data item can only be read

lock primitive tell us what we can do with the data, but doesn't tell us what other can do

2. 🌱 **Lock compatibility matrix**
 - what other people can do, when you having lock yourself
 - if you have S-lock other want S-lock → then true → yess



Lock-Based Protocols (Cont.)

▪ Lock-compatibility matrix

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
 - Any number of transactions can hold shared locks on an item,
 - But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.
-
- false → the one who request later have to wait

Schedule With Lock Grants		
- Grants omitted in rest of chapter - Assume grant happens just before the next instruction following lock request	T_1	T_2
	concurrency-control manager	
	lock-X(B)	
	read(B)	
	$B := B - 50$	
	write(B)	
	unlock(B)	
		lock-S(A)
		read(A)
		unlock(A)
		lock-S(B)
		read(B)
		unlock(B)
		display(A + B)
	lock-X(A)	
	read(A)	
	$A := A + 50$	
	write(A)	
	unlock(A)	

some example from a book (that just lock primitive and lock compatibility matrix does not work) we need lock protocol → not serializable

- the problem with this is → old A old B or new A new B now it is one old one new → inconsistency analysis problem.
- this is not 2PL.

3. Lock Protocol

gauruntee conflict or view serializable schedule

- rule which instruct when to apply lock and unlock instruction.
- locking protocol is a set of rules followed by all transactions while requesting and releasing locks
- locking protocol enforce serializability by restricting the set of possible schedule.

Popular Lock Protocol → 2 phase locking protocol (the most popular one) (2PL)

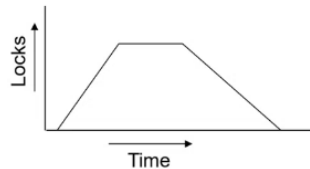
→ 2 Phase Locking Protocol

gauruntee conflict serializable schedule.



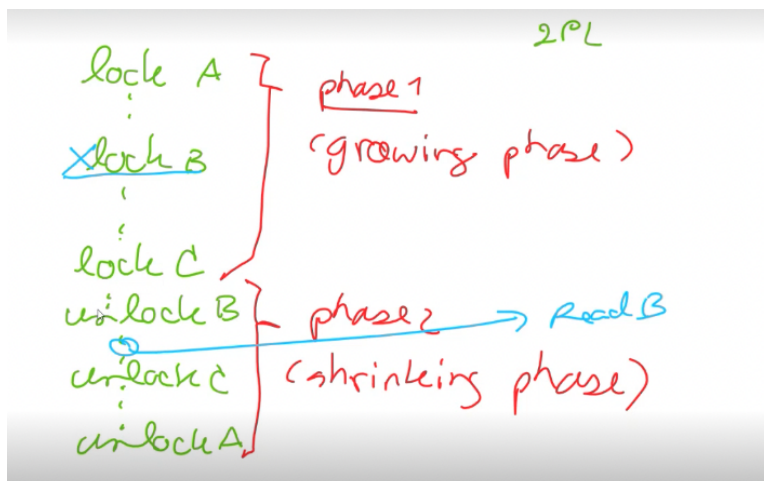
The Two-Phase Locking Protocol

- A protocol which ensures conflict-serializable schedules.
- Phase 1: **Growing Phase**
 - Transaction may obtain locks
 - Transaction may not release locks
- Phase 2: **Shrinking Phase**
 - Transaction may release locks
 - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).



- protocol what ensure conflict serializable schedule
 - the scheduler will always produce conflict serializable schedule.
- 1. Growing phase → lock only no unlock.
- 2. Shrinking phase → once unlock, no more locks.

(concept → if you unlock → no more locking)



- Suppose, B is X-lock, and someone read B there, and later transaction failed, the one who read B will obtain the wrong result.



2PL guarantee only conflict serializable schedule, not cascadeless schedule

Not 2PL version

T_1	T_2	concurrency-control manager
lock-X(B)		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		grant-X(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

- T_1 x lock B , and after update unlock B (not 2PL)

2 PL version (lead to deadlock)

- Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

- Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .
- Such a situation is called a **deadlock**.
 - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

- T_3 x lock B but after update not unlock B yet, because it has to lock everything first, and then T_2 share lock a , read a , not yet unlock A , try to share lock B , but B has x lock B
 - this share lock has to wait
 - T_4 wait for B

T3 wait for T4, T4 wait for T3 → **dead lock**

- 2PL turn schedule that are not conflict serializable to be deadlock → it can't continue proceed.

More Example

(Locking but not 2PL)

- a schedule money transfer transaction with locking (not 2PL)

```
T1: lock-X(B);  
    read(B);  
    B := B - 50;  
    write(B);  
    unlock(B);  
    lock-X(A);  
    read(A);  
    A := A + 50;  
    write(A);  
    unlock(A).
```

Figure 18.2 Transaction *T*₁.

(2PL ver)

```
T3: lock-X(B);  
    read(B);  
    B := B - 50;  
    write(B);  
    lock-X(A);  
    read(A);  
    A := A + 50;  
    write(A);  
    unlock(B);  
    unlock(A).
```

Figure 18.5 Transaction *T*₃ (transaction *T*₁ with unlocking delayed).

(2PL ver)

```

T4: lock-S(A);
    read(A);
    lock-S(B);
    read(B);
    display(A + B);
    unlock(A);
    unlock(B).

```

Figure 18.6 Transaction T_4 (transaction T_2 with unlocking delayed).

(Deadlock)

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

Figure 18.7 Schedule 2.

- T3 wait for T4, and T4 wait for T3
 - T4 want to issue a lock-S(B), but B is lock by T3

DeadLock

- A system is in a deadlock state if there exist a set of transactions such that every transaction in the set is waiting for another transaction in the set.
(pg.849)



Deadlock

- Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

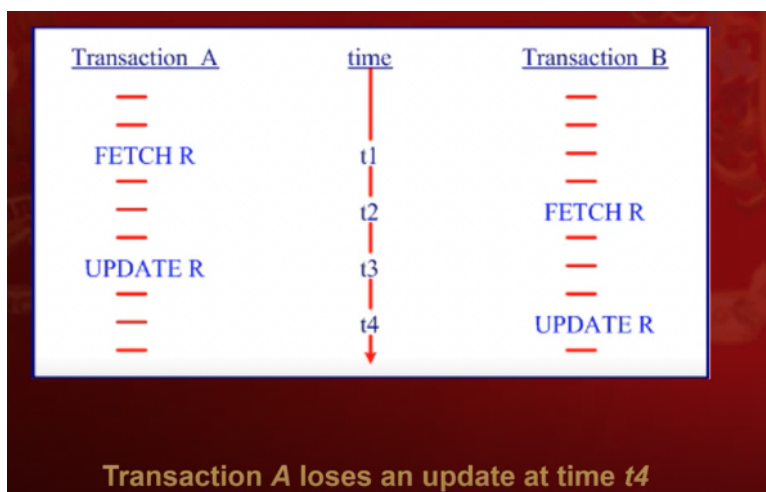
- Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .
- Such a situation is called a **deadlock**.
 - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

- in practice, one of them will be time-out and let the other one go.
- Each transaction must have timestamp or priorities
 - maybe the one older get timeout first or the one with less priority that get timeout first

in 2PL, once you start unlocking, unlocking, there are 2PL

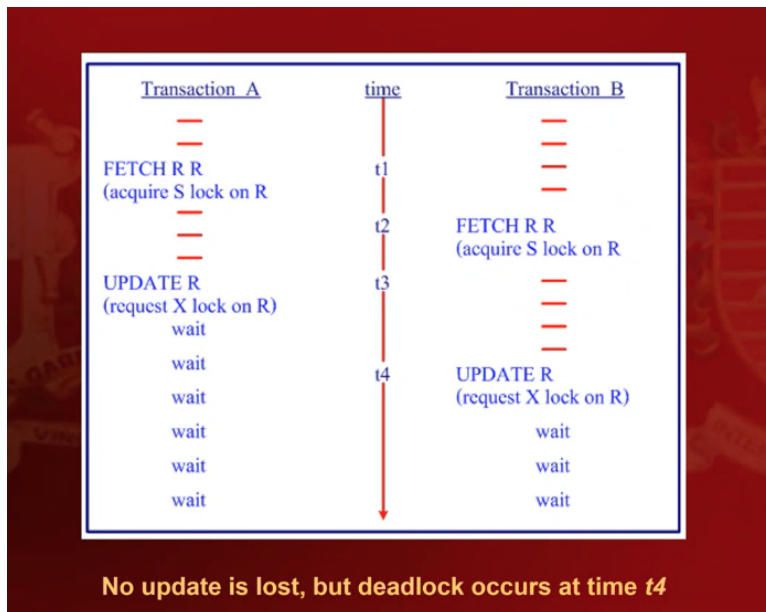
What about we apply 2PL to concurrency control problem

- lost update problem - if you don't do anything, you obtain incorrect result.
 - if you try locking without protocol, you still obtain incorrect result.



- you apply 2 PL

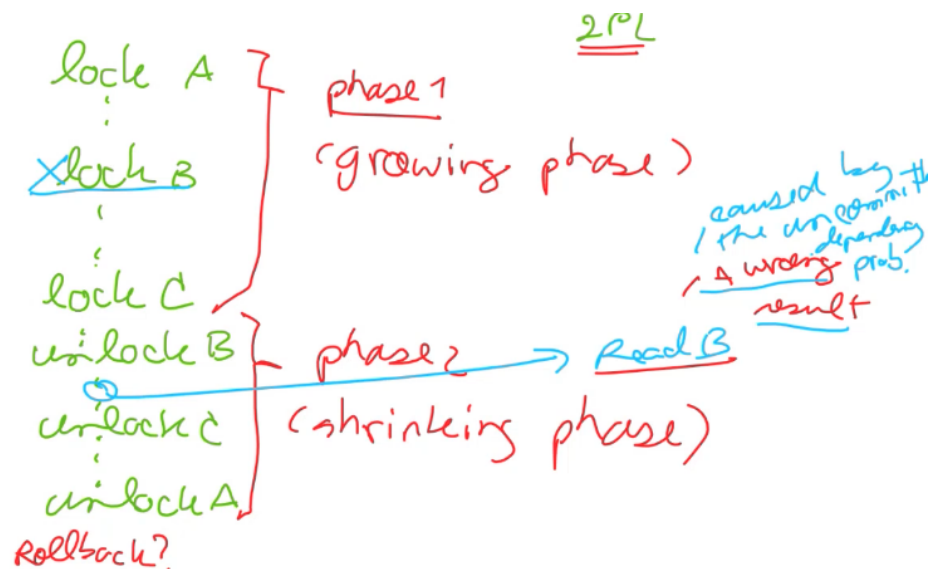
- lock upgrade is upgrade from S lock to X lock



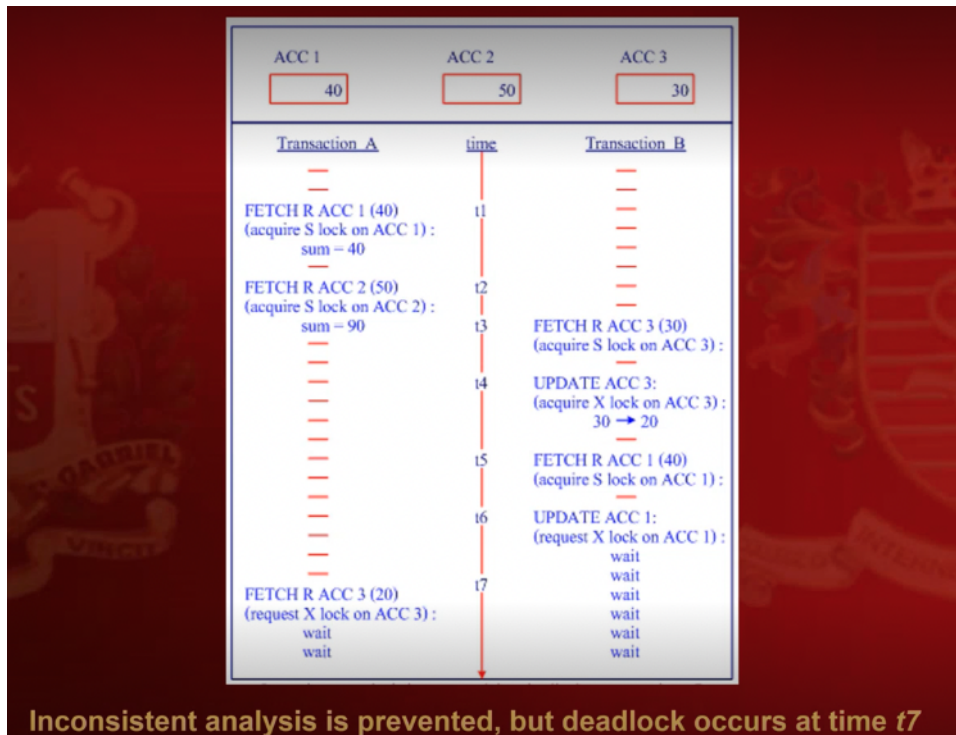
- A wait for B, transaction B also wait for A to update R
- A wait for B, B wait for A → deadlock

2. uncommitted dependency problem

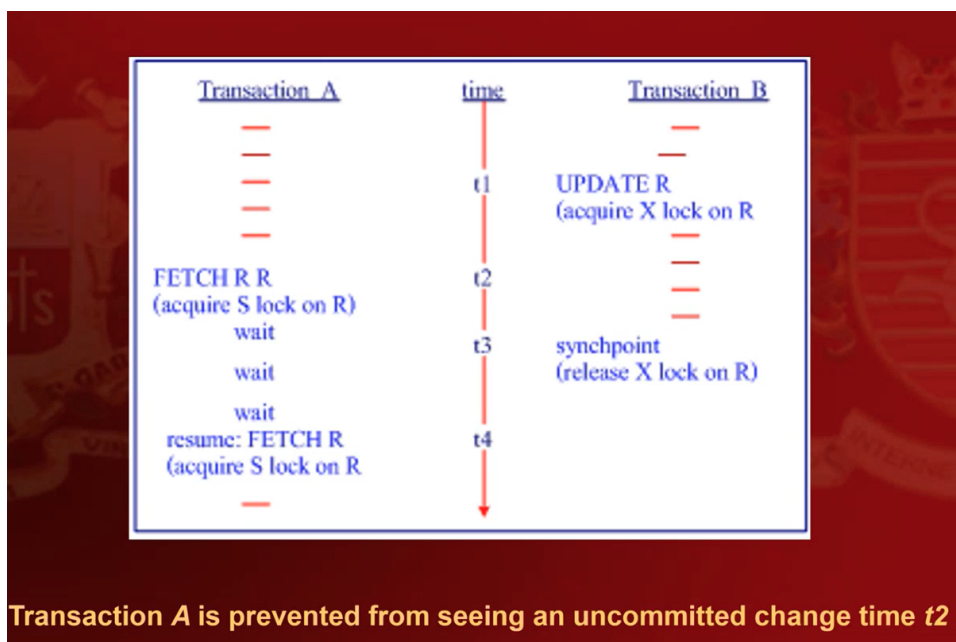
- with just 2PL, you can still have a wrong result cause by the uncommitted dependency problem.
- what if someone want to read B → they can (what if later transaction failed) → that one obtain wrong result
- we unlock B to soon



3. inconsistent analysis problem



- could be a deadlock, B want to update account 1, but can't because account 1 is lock by A
- similarly, A try to read account 3, but it has been lock by B



- A try to fetch R, A has to wait → B not yet commit
- X lock wait until sync point for it to be released

with 2PL, you can still get wrong result caused by uncommitted dependency prob + it not cascadeless too



2PL is not enough, not guarantee cascadeless schedule.

- 2PL guaranteed conflict serializable schedule, but the point is conflict serializable doesn't consider a case of failure. So there could be problem (uncommitted dependency problem)

How to solve uncommitted dependency problem? How to guarantee the cascadeless schedule? (the same thing)

(unlock too soon → problem is read after unlock before commit)

- solution → we should not allow unlock of X-lock before commit
 - we need other condition
 - **2PL + xlock must be unlocked at the sync point only**
- S Lock can be unlock anytime in shrinking phase

Strict 2PL

| This is called the strict 2PL protocol (the most popular one)

However, after implementation, some DBMS have to figure out which lock is x-lock and which is s-lock, and they have to handle it separately → more work to DBMS

So, there is more version

Rigorous 2PL

- Rigorous 2PL all lock released at sync point



- 

How to handle deadlock?(how to obtain correct result without deadlock)

<u>Transaction A</u>	<u>time</u>	<u>Transaction B</u>
—	t1	—
—		—
FETCH R R (acquire S lock on R)	t2	—
—		—
—	t3	FETCH R R (acquire S lock on R)
UPDATE R (request X lock on R)		—
wait	t4	—
wait		—
wait		—
wait		—
wait		UPDATE R (request X lock on R)
wait		wait
wait		wait
wait		wait

12

- form change surname, address → change go to different HR scenario example
 - if we don't do anything → there are in the same row → read old record, modify(they both, only B succeed)
 - if you apply 2PL, it will be deadlock.
- Solution → group data item together (merge them into 1 transaction)
 - someone check form and group the form belong to a person
 - what if they don't want to get someone to check and group the form.
- Solution
 - use X-lock from the beginning to turn potential deadlock into a wait → wait is better than Deadlock
 - if you write program and lock by yourself → it's you who do this.
 - but database environment where you don't do this locking by yourself, you have to tell locking to do so. When you open cursor → this mode is called wait mode
 - select then update, read before write → select then update mode → this is share lock in the beginning, and later promote to be X-lock
 - in practice, you open cursor in sql for wait mode or select and update mode

3. Inconsistent Analysis

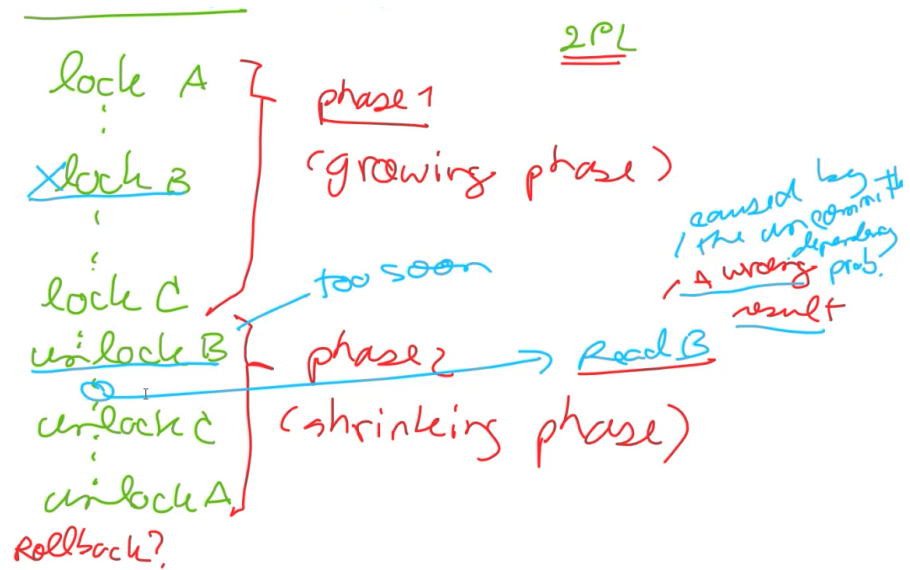
- 2PL → deadlock
- To avoid deadlock
 - first solution → group transaction that work on the same object together
 - use the wait mode to apply x-lock from the very beginning instead of the s-lock from the very beginning of the transaction (this will turn deadlock into a wait) (select for update mode, wait mode)
 - if you the one who develop this tool → simply x-lock from the very beginning
 - if you the one who use this tool → simply refer to wait mode or select for update mode

when we consider this problem, we consider 1 row at a time (we think of R as a row)

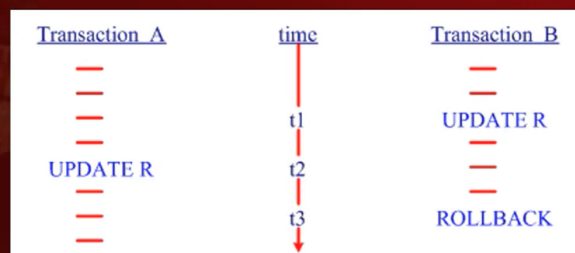
Let consider something else, what if this R is not just 1 row

- airline seat booking, cinema → customer book few seat → many seat available.
 - if you lock the entire R for the line of customer, the entire line wait → what should be a solution
 - you have more than 1 line → now they all wait
 - ??.....What if R is not a row, but many rows. eg. seat allocation? How to avoid long waits.

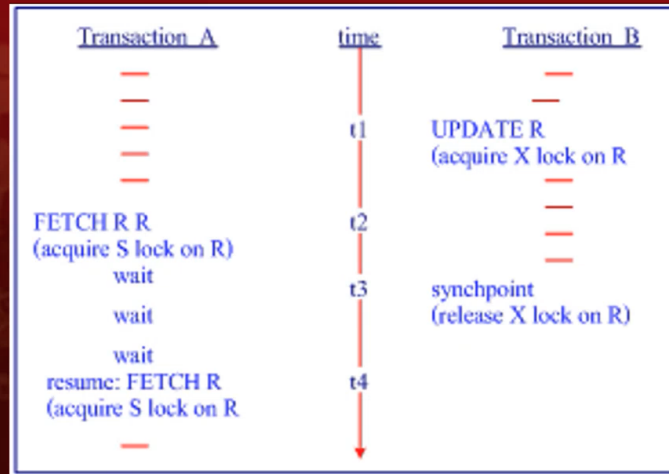
2. The uncommitted dependency problem



Transaction A becomes dependent on an uncommitted change at time t_2



Transaction A updates an uncommitted change at time t_2 , and loses that update at time t_3 .



Transaction A is prevented from seeing an uncommitted change time t2

- x-lock must be release at the sync point.
 - Strict 2PL or rigorous 2PL
3. The inconsistent analysis problem (read only and update)
- do nothing → incorrect result
 - lock without protocol → incorrect result
 - 2 PL → strict not strict doesn't matter, it all lead to deadlock
 - lock a larger object such as table → wait instead of deadlock (**multiple granularity**)
 - granularity → the size of the data which is to be locked.
 - no wait no deadlock??? → oracle and postgres using them
4. The phantom phenomenon (phantom insert) (read only and insert)
- this one you can use **multiple granularity**
 - because when you lock the entire table, no body else can insert or update

Prob3 and 4

- what if we want correct results, no waits, no deadlocks, how?

✓ Multiple granularity

- multiple granularity has its own problem
- how do you know the entire table is available for you to lock
- How to check the availability of each granularity
 - you could scan the whole thing → nooooo, tedious
- many techniques are proposed and a well-known solution
 - solution → use the intention locks mode to check the availability of granularity
 - intention lock modes



Intention Lock Modes

- In addition to S and X lock modes, there are three additional lock modes with multiple granularity:
 - **intention-shared (IS)**: indicates explicit locking at a lower level of the tree but only with shared locks.
 - **intention-exclusive (IX)**: indicates explicit locking at a lower level with exclusive or shared locks
 - **shared and intention-exclusive (SIX)**: the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive-mode locks.
 - Intention locks allow a higher level node to be locked in S or X mode without having to check all descendent nodes.
-
- IS - if a node is locked → subtree under it is being shared lock by someone
 - IX - if a node with IX → subtree under it is being either Slock or Xlock

Some example

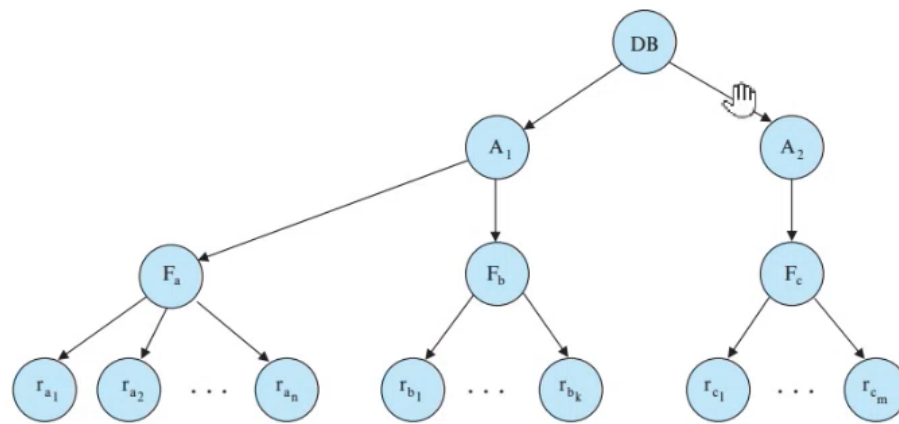
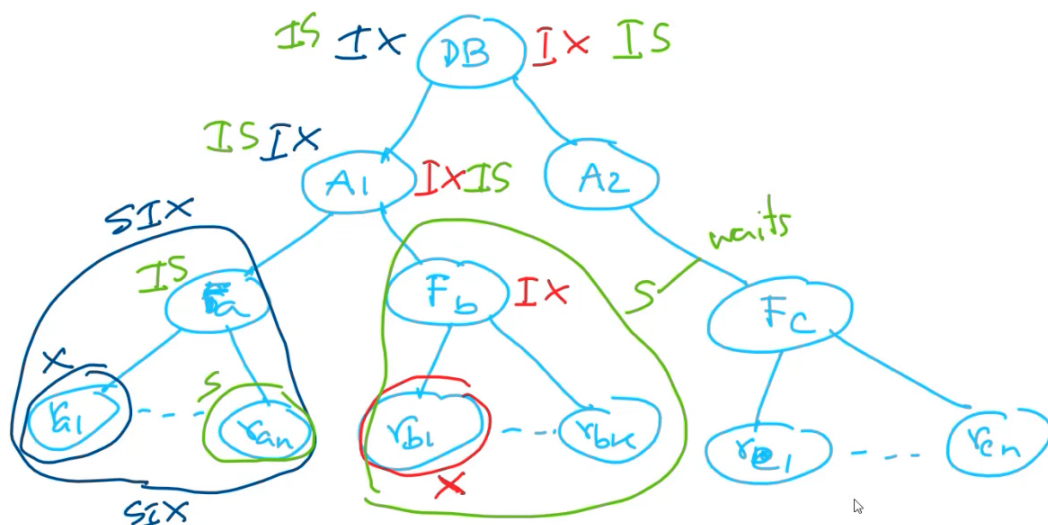


Figure 18.15 Granularity hierarchy.

- the levels starting from the coarsest (top) level are
 - database, area, file, record
- more lock primitive, not just S and X as usual



SIX → select for update

- false mean the one come later → wait

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

Figure 18.16 Compatibility matrix.



Multiple Granularity

- Allow data items to be of various sizes and define a hierarchy of data granularities, where the small granularities are nested within larger ones
- Can be represented graphically as a tree (but don't confuse with tree-locking protocol)
- When a transaction locks a node in the tree *explicitly*, it *implicitly* locks all the node's descendants in the same mode.
- Granularity of locking (level in tree where locking is done):
 - **Fine granularity** (lower in tree): high concurrency, high locking overhead
 - **Coarse granularity** (higher in tree): low locking overhead, low concurrency

◦ generic question, which one is better lock smaller one or bigger one

- smaller one allow more sharing, but there are people that said lock big one could be better in some circumstances(good reason too)
 - it easier to lock big object, in fact, we lock data in the main memory not a disk.
 - if we lock 1 million rows → we need 1 million switches on/off(per row) for each and every row → great effort to handle that
 - lock bigger object require less effort
 - intuitively, we should saml one, allow concurrent access
 - from experiment, when busy → lock small one
 - so based on circumstances

- for deadlock avoidance, and better throughput.

Multiple granularity (usefulness of it)

- Avoid deadlocks when solving the inconsistent analysis and the phantom phenomenon.
- reduce locking overhead at off peak.

Multiple Granularity

- Avoid deadlocks when solving the inconsistent analysis problem and the phantom phenomenon.
- Reduce locking overheads at off peak.

TA Select sum (amount) From Account where branch = 'Ladkrabang'	TB update ACC3 $\xrightarrow{+10}$ ACC1
---	---

- lock only ladkrabang row (Fa could be row from ladkrabang branch)
 - F is result from sql statement

TA Select sum (amount) From Account where branch = 'Ladkrabang'	TB update ACC3 $\xrightarrow{+10}$ ACC1
---	---

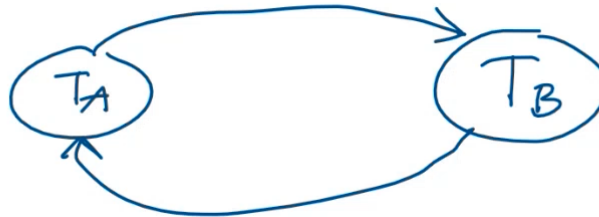
lock only the result rows
of this statement

→ Command-level granularity

→ Command-level granularity - if lock only the result rows of this statement.

Deadlock Handling(p.849)

Deadlock → wait-for graph



- A wait for B, and B wait for A
- wait-for graph that has a circle is a deadlock
- **Deadlock prevention**
 - either allow younger transaction wait for older one, older one wait for younger one (only 1 side)
 - younger transaction can wait or older transaction can wait
 - like allow one, if other wait → kill that transaction

there 2 scheme for deadlock prevention



More Deadlock Prevention Strategies

- **wait-die** scheme — non-preemptive
 - Older transaction may wait for younger one to release data item.
 - Younger transactions never wait for older ones; they are rolled back instead.
 - A transaction may die several times before acquiring a lock
- **wound-wait** scheme — preemptive
 - Older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it.
 - Younger transactions may wait for older ones.
 - Fewer rollbacks than *wait-die* scheme.
- In both schemes, a rolled back transactions is restarted with its original timestamp.
 - Ensures that older transactions have precedence over newer ones, and starvation is thus avoided.

read in here too

- non-preemptive → the process keep the resource until the end, it won't release before the process finish

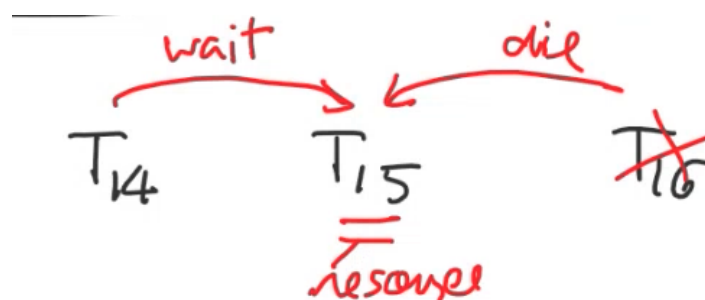
- preemptive → the process could release the result

1. The **wait-die** scheme is a nonpreemptive technique. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp smaller than that of T_j (i.e., T_i is older than T_j). Otherwise, T_i is rolled back (dies).
For example, suppose that transactions T_{14} , T_{15} , and T_{16} have timestamps 5, 10, and 15, respectively. If T_{14} requests a data item held by T_{15} , then T_{14} will wait. If T_{16} requests a data item held by T_{15} , then T_{16} will be rolled back.
2. The **wound-wait** scheme is a preemptive technique. It is a counterpart to the wait-die scheme. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if it has a timestamp larger than that of T_j (i.e., T_i is younger than T_j). Otherwise, T_j is rolled back (T_j is *wounded* by T_i).
Returning to our example, with transactions T_{14} , T_{15} , and T_{16} , if T_{14} requests a data item held by T_{15} , then the data item will be preempted from T_{15} , and T_{15} will be rolled back. If T_{16} requests a data item held by T_{15} , then T_{16} will wait.

- Deadlock prevention schemes

1. wait-die (non preemptive) → older transaction wait

- Suppose T15 has resources
- T14 older one, want resources (wait for T15)
 - wait until T15 finish and release the result
- if T16 younger one want to wait → T16 die



- therefore, not possible to have deadlock

2. wound-wait (preemptive) → younger transaction wait

- T15 release the resource before normal termination



The thing is when we use this → some transaction die unnecessary (maybe it not the cause of deadlock, but we kill txs to prevent the deadlock)

- this is why detection is more popular than prevention

- **Deadlock detection**

- detect the circle in a graph → once cycle is found, then it is a deadlock
- actual problem in practice, is when to check, like we have algo to check but when
 - when should detection take place.
 - you can't check everytime you have wait → that too much
 - when the respond time is too slow → check manually
 - once, cycle is found, break them manually
 - semiautomate, manual

Transaction Definition in SQL



Transaction Definition in SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
 - **Commit work** commits current transaction and begins a new one.
 - **Rollback work** causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
 - Implicit commit can be turned off by a database directive
 - E.g., in JDBC -- `connection.setAutoCommit(false);`
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
 - E.g. In SQL **set transaction isolation level serializable**
 - E.g. in JDBC -- `connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE)`

Conclusion

Lock-based Technique

- Prob 1 → Prob4: A strict or rigorous 2PL
 - strict → x lock is release at sync point
 - rigorous → all locks are released at sync point (prob2 solved)
- Prob1
 - group transaction that refer to the same object together
 - X lock from the beginning
- Prob 2
 - employ strict or rigorous 2PL
- Prob3 and 4
 - Multiple granularity

A lock-based system requires: strict or rigorous 2PL + Multiple granularity with intention lock

**A problem: lock-based systems may lead to dead locks.
What if we don't want deadlock at all.**

- there other idea → simply if you don't want deadlock, don't employ any lock
 - if we don't lock at all, there will not be any deadlocks.
 - **Timestamp based techniques - use time checking to arrange conflict serializable schedule**